

Steganographic File Integrity Checker

Name: Khushi Patwa

Intern id :234

What is a Steganographic File Integrity Checker?

A Steganographic File Integrity Checker is a security tool or system that ensures a file has not been tampered with, by using a combination of Data security and integrity are foundational in digital communications. Ensuring that files remain untampered during transfer or storage is critical in many domains such as forensics, legal documentation, and secure communications.

This PoC presents a novel approach for securing files using SHA256 hashing combined with LSB steganography, embedding file integrity data within a seemingly innocent image. The embedded cryptographic hash can later be extracted to verify the original file's authenticity.

- **Cryptographic hashing** (like SHA256) – to generate a digital fingerprint of a file.
- **Steganography** – to hide that fingerprint (hash) *inside an image* so it's not visible or easily detected.

File Integrity Checker

- This checks whether a file has changed or been corrupted.
- It uses a cryptographic hash function (like SHA256) to create a unique digital signature of the file.
- If even 1 bit of the file changes, the hash will be completely different.

Steganography

- The science of hiding secret data inside other non-secret files.
- In this case, the file's hash is hidden inside an image file using a method like Least Significant Bit (LSB) steganography.
- The image looks normal to the human eye, but it secretly contains hidden data.

How Does It Work? Step-by-Step

Step 1: Generate File Hash

- Input: Original file (e.g., document.pdf).
- Compute SHA256 hash → a 64-character hexadecimal string representing the file fingerprint.

- Convert this hash to binary to prepare for embedding.

Step 2: Embed Hash into Image

- Input: Cover image (e.g., cover.png).
- Use LSB steganography to modify the least significant bit of the image pixels.
- Embed the hash bit-by-bit in the pixel data.
- Output: A stego image visually indistinguishable from the original but containing the hidden hash.

Step 3: Distribute or Store

- The stego image can be shared, stored, or transmitted safely.
- The hash is concealed, preventing casual attackers from knowing or altering it.

Step 4: Verification (Later Point)

- Extract the hidden hash from the stego image by reading the LSBs.
- Compute the hash again from the target file.
- Compare the two hashes:
 - If they match → File is **authentic and unchanged**.
 - If not → File has been **tampered or corrupted**.

Purpose

The purpose of a Steganographic File Integrity Checker is to:

- Ensure that files are authentic and unchanged.
- Keep the verification data (hash) hidden, so it's not obvious or easily altered.
- Provide a covert integrity check, useful in forensics, secure transfers, or anti-tamper systems.

Conclusion

This PoC offers a lightweight, effective method to validate file integrity using **hashing + steganography**. By hiding hashes in image files, it removes the need to store or transmit them separately, while still enabling full integrity verification. It's an ideal solution for secure environments where subtle file tampering needs to be detected with minimal overhead.